

Tables and tabular data

Tables are for data that makes sense in rows and columns. Think of an orders listing, weekly schedule, a price list, plan comparisons, sports standings, or financial figures. If the relationship between rows and columns matters, use a table.

For a long time, developers also used tables to arrange whole pages. That is not how layouts are built now. CSS handles layout; tables are for data.

The easiest test is this: can you give each column a label, and does each row describe one item using those same labels? If yes, a table is probably a good fit. If you are only trying to place boxes next to each other, use CSS layout instead.

The basic structure

A minimal table looks like this:

html

```
<table>
  <tr>
    <th>Plan</th>
    <th>Price</th>
    <th>Users</th>
  </tr>
  <tr>
    <td>Starter</td>
    <td>$9</td>
    <td>1</td>
  </tr>
  <tr>
    <td>Team</td>
    <td>$29</td>
    <td>5</td>
  </tr>
</table>
```

The pieces are:

- `<table>` is the container for the whole thing.
- `<tr>` is a table row.
- `<th>` is a header cell. Browsers usually render it bold and centered, but the meaning is "this cell labels a row or column".
- `<td>` is a data cell. This is where the table's values go.

Cells go inside rows. Rows go inside the table. That is the whole shape.

Plan	Price	Users
Starter	\$9	1
Team	\$29	5

The important part is the relationship between the cells. In the example above, **Starter** , **\$9** , and **1** are not three random values. They are one row about the same plan. The headers explain what each value means.

Captions, head, and body

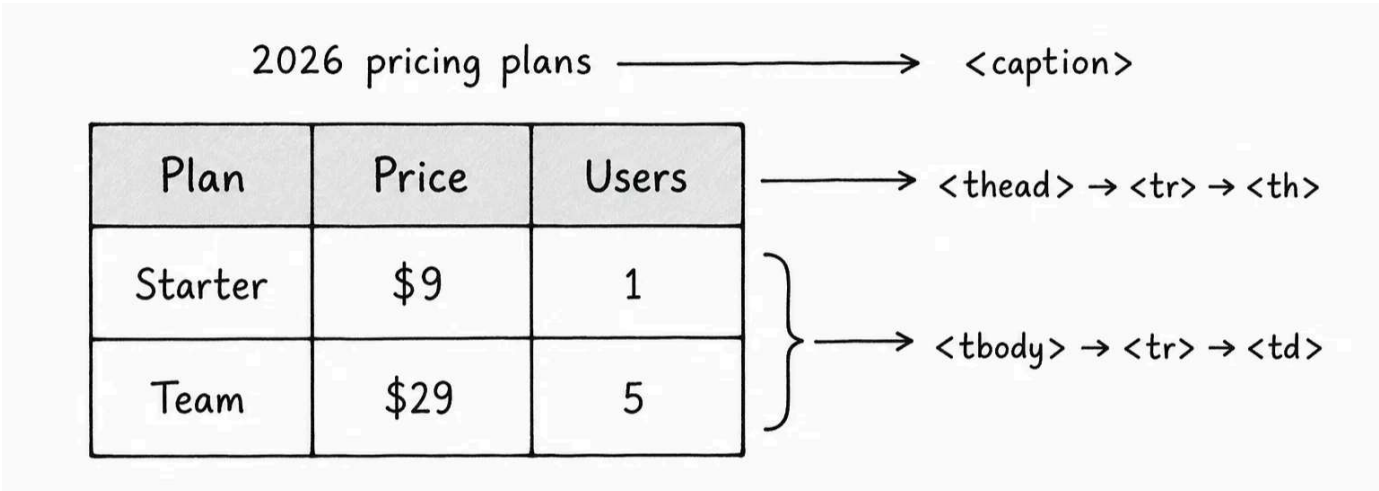
A practical table usually benefits from three more elements.

`<caption>` is the table's title. It belongs directly inside `<table>` as the first child.

`<thead>` and `<tbody>` group the header rows and data rows. You can make a table without them, but adding them makes the structure clearer and easier to style, print, scan, and maintain. There is also `<tfoot>` for a footer row, often used for totals.

html

```
<table>
  <caption>2026 pricing plans</caption>
  <thead>
    <tr>
      <th>Plan</th>
      <th>Price</th>
      <th>Users</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Starter</td>
      <td>$9</td>
      <td>1</td>
    </tr>
    <tr>
      <td>Team</td>
      <td>$29</td>
      <td>5</td>
    </tr>
  </tbody>
</table>
```



`<thead>` and `<tbody>` do not make the table look dramatically different on their own. Their job is structure. They help browsers, screen readers, CSS, and future maintainers understand which rows are labels and which rows are data.

The scope attribute

When a header cell labels a column, set `scope="col"`. When it labels a row, set `scope="row"`. This makes the relationship between headers and data cells explicit.

html

```
<thead>
  <tr>
    <th scope="col">Plan</th>
    <th scope="col">Price</th>
  </tr>
</thead>
<tbody>
  <tr>
    <th scope="row">Starter</th>
    <td>$9</td>
  </tr>
</tbody>
```

For very simple tables, that relationship is often easy to infer. Once row headers enter the picture, setting `scope` explicitly is the clearer habit.

This is especially useful for screen reader users. Instead of hearing a long stream of disconnected values, they can understand each data cell in relation to its row and column headers.

Spanning cells

A cell can span more than one column or row.

html

```
<tr>
  <th colspan="2">Pricing</th>
</tr>
```

`colspan="2"` makes this cell take up two columns. `rowspan="3"` would make a cell span three rows. Use these when the table really needs a merged

cell, such as a section header above two columns, not as a styling shortcut.

```
<table>
  <tr>
    <th colspan="3">Monthly Report</th>
  </tr>
  <tr>
    <th>Product</th>
    <th>Q1</th>
    <th>Q2</th>
  </tr>
  <tr>
    <td rowspan="2">Product A</td>
    <td>120</td>
    <td>150</td>
  </tr>
  <tr>
    <td>130</td>
    <td>160</td>
  </tr>
</table>
```

→

Monthly Report		
Product	Q1	Q2
Product A	120	150
	130	160

⚠ **Keep tables simple** - Complex tables with multiple header levels, merged cells, or nested sections can become hard to read and hard to make accessible. Use `colspan` and `rowspan` when the data truly needs them, but prefer a simpler table when you can.

Tables are for data, not layout

It used to be common to build page layouts with tables: invisible grids of rows and columns holding navigation, content, and ads. That causes problems:

- **Tables describe data relationships.** Screen readers and search engines treat tables as data. If you use them for layout, the structure no longer matches the content.
- **Layout changes are hard inside tables.** Responsive design, where the page rearranges itself on phones, is awkward to express with table cells.
- **CSS layout tools exist now.** Flexbox and grid handle every layout job tables used to be stretched to cover.

Reserve `<table>` for data that actually has rows and columns.

Good table use:

- A pricing comparison where each row is a plan and each column is a feature.
- A schedule where each row is a day and each column is a detail about that day.
- A report where each row is a record and each column is a field.

Poor table use:

- A page header with a logo on the left and navigation on the right.
- A grid of product cards.
- A two-column article layout.

When tables come from app data

Real apps often generate tables from data: admin dashboards, reports, exports, invoices, schedules, and search results.

A few habits worth picking up:

- **Include a caption** for tables generated dynamically. It gives the table a clear purpose before the user starts navigating the cells.
- **Group header rows clearly.** Use `<thead>` for headers. Printing, sticky-header CSS, and future styling are easier to manage when header rows are grouped clearly.
- **Avoid rendering huge tables all at once.** Paginate, filter, or summarize large datasets instead of sending thousands of rows in one response. Huge tables are slow to load and hard to use.

Try it

Build a small weekly schedule. Save this as `schedule.html` and open it in your browser:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Weekly schedule</title>
  </head>
  <body>
    <h1>This week</h1>

    <table>
      <caption>Study schedule</caption>
      <thead>
        <tr>
          <th scope="col">Day</th>
          <th scope="col">Topic</th>
          <th scope="col">Hours</th>
        </tr>
      </thead>
      <tbody>
        <tr>
          <th scope="row">Monday</th>
          <td>HTML basics</td>
          <td>2</td>
        </tr>
        <tr>
          <th scope="row">Tuesday</th>
          <td>Links and forms</td>
          <td>3</td>
        </tr>
        <tr>
```

```
<th scope="row">Wednesday</th>
<td>Practice project</td>
<td>2</td>
</tr>
</tbody>
</table>
</body>
</html>
```

Add a few more rows. Try changing one `<th scope="row">` to a `<td>`. The day no longer acts like a row label; it becomes just another value. Open DevTools and expand the `<table>` node in Elements. You should see the same `thead`, `tbody`, and `tr` structure you wrote.

< 0 / 6 >

✔Knew

0 Items

☆Learnt

0 Items

⏮Skipped

0 Items

↺ResetProgress

Test Your Knowledge

When should you use a table?

[Click to Reveal the Answer](#)

✔Already Know that

☆Didn't Know that

⏮Skip Question

✓

LESSON COMPLETE

Nicely done.

Up next: Pricing Comparison Table

Next Lesson